This document serves as the **Technical Specification Reference** for the Frontend Development team. It details the UI requirements, data visualization strategies, and the API data contracts required to build the Admin Dashboard for the NDRF VR Simulation.

This document is also formatted to be ingested by an AI assistant to understand the context of the application structure.

Project: NDRF Command Center (Admin Dashboard)

Technical Specification v1.0

1. High-Level Architecture

- **Frontend:** React.js / Next.js (Recommended for Dashboard UI).
- **Backend:** Python Flask.
- **Database:** SQL (PostgreSQL/MySQL).
- **Data Flow:**
  1. **VR Simulation** pushes raw session data (JSON) $\rightarrow$ **Flask API**.
  2. **Flask API** normalizes data and stores it in **SQL Tables** (Users, Sessions, Metrics, Failures).
  3. **Admin Dashboard** polls/requests aggregated data from **Flask API** for visualization.

---

2. Page 1: Overview (Mission Control)

Goal: High-level operational awareness. A quick glance for the Commander to see training velocity and pass rates.

2.1. UI Components

1. **KPI Cards (Top Row):**
   - **Total Sessions Conducted** (Integer).
   - **Weekly Pass Rate** (Percentage + Trend Indicator $\uparrow$/$\downarrow$).
   - **Active Trainees** (Count of unique users trained this week).
   - **Avg. Mission Duration** (MM:SS).
2. **Weekly Activity Graph (Line Chart):**
   - X-Axis: Days (Mon-Sun).
   - Y-Axis: Number of Sessions.
   - Lines: Total Sessions vs. Failed Sessions.

3. **Success Rate by Trainee Rank (Bar Chart/Pie Chart):**
    - Breakdown of pass rates by Squad Leaders vs. Rescuers.

2.2. API Contract: GET /api/dashboard/overview

JSON

```
{
  "kpi": {
    "total_sessions": 142,
    "weekly_pass_rate": 78.5,
    "pass_rate_trend": "UP", // or "DOWN"
    "active_trainees": 34,
    "avg_duration_sec": 750
  },
  "weekly_activity": {
    "labels": ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"],
    "total_sessions": [10, 15, 12, 20, 18, 5, 2],
    "failed_sessions": [2, 3, 1, 5, 2, 0, 0]
  },
  "success_by_rank": [
    { "rank": "Squad Leader", "pass_rate": 85 },
    { "rank": "Rescuer", "pass_rate": 70 }
  ]
}
```

---

3. Page 2: Personnel Management

Goal: CRUD operations for NDRF officers.

3.1. UI Components

1. **Personnel Table:**
    - Columns: ID, Name, Rank, Total Sessions, Last Trained Date, Status (Active/Inactive).
    - **Action Column:** Edit Button (Modal), View History Button (Redirects to Reports filtered by User).
2. **"Add Personnel" Button:** Opens a modal to create new users.

3.2. API Contract: GET /api/personnel

Note: Include pagination parameters ?page=1&limit=20.

JSON

```
{
  "total_count": 45,
  "users": [
    {
      "user_id": "NDRF_001",
      "name": "Rahul Singh",
      "rank": "Squad Leader",
      "total_sessions": 12,
      "last_trained_date": "2024-12-08",
      "status": "ACTIVE"
    }
  ]
}
```

3.3. Head Space for Search Implementation

- **Requirement:** Frontend must implement a debounce search bar calling the backend.
- **Backend Hook:** GET /api/personnel?search={query}
- **SQL Logic:** SELECT * FROM users WHERE name ILIKE %query% OR user_id ILIKE %query%.

---

4. Page 3: Analytics (The "Deep Dive")

Goal: The Selling Point. Detailed breakdown of specific Hazmat skills across the entire battalion.

4.1. UI Components

1. **Zone Precision Bell Curve (Line/Area Chart):**
    - Visualizes the aggregate accuracy of cone placement.
    - X-Axis: PPM Deviation (Ideal is 0).
    - Y-Axis: Number of Trainees.
    - *Insight:* Are most trainees placing cones too conservatively (safe but slow) or

dangerously (too close to source)?

2. **Triage Efficiency Matrix (Bar Chart):**
    - Comparison of "Correct Order" vs "Incorrect Order" in rescues.
3. **Failure Type Breakdown (Horizontal Bar Chart):**
    - Top reasons for mission failure (e.g., "PPE Violation" vs "Zone Breach").

4.2. API Contract: GET /api/analytics/aggregate

JSON

```
{
  "zone_precision_stats": {
    "average_ppm_deviation_warm": 1.5, // Avg distance from ideal 0.5ppm
    "average_ppm_deviation_hot": 4.2,  // Avg distance from ideal 10ppm
    "most_common_error": "UNDERESTIMATION" // Placed cone too deep in gas
  },
  "triage_stats": {
    "perfect_sequence_rate": 65.0, // % of sessions with Perfect Order
    "black_victim_error_rate": 10.0 // % of sessions where dead victim was moved
  },
  "top_failures": [
    { "type": "ZONE_VIOLATION_WARM", "count": 45 },
    { "type": "OXYGEN_DEPLETION", "count": 12 },
    { "type": "TRIAGE_PROTOCOL_ERROR", "count": 8 }
  ]
}
```

---

5. Page 4: Reports (Individual Session Detail)

Goal: The core feedback loop. Displays the specific session data sent from the Game, enriched with AI analysis.

5.1. Session List (Master View)

- Table listing all recent sessions.
- Filters: Date Range, Pass/Fail, User Name.

5.2. Session Detail View (The "Report Card")

This view renders when a specific Session ID is clicked. It must visualize the JSON data received from the game.

Component A: Header

- Session ID, Date, Trainee Name.
- **Time Clock:** Display 12:45 (Color coded: Green if <15min, Red if >15min).

Component B: Zone Demarcation Visualization

- **Visual:** Two Horizontal Gradient Bars.
  1. **Cold $\rightarrow$ Warm:** Mark "Ideal" at 0.5 PPM. Place a "Pin" at the user's at_cone value.
  2. **Warm $\rightarrow$ Hot:** Mark "Ideal" at 10.0 PPM. Place a "Pin" at the user's at_cone value.
- *Data Source:* zone_demarcation array from Game JSON.

Component C: Triage Chain

- **Visual:** A flowchart of the rescue order.
  - Example: [🔴 Victim A] $\rightarrow$ [🟢 Victim B] $\rightarrow$ [⚫ Victim C].
- *Validation:* Backend sends is_optimal_order: true/false. If false, Frontend highlights the incorrect step in Red.

Component D: AI Analysis (Markdown Render)

- **Requirement:** Frontend must use a Markdown renderer (e.g., react-markdown).
- **Content:** The backend will generate a text summary using an LLM based on the game metrics and send it as a string.

5.3. API Contract: GET /api/reports/{session_id}

This endpoint combines SQL data (User details) + Game Data (JSON Blob).

JSON


```
{
  "metadata": {
    "session_id": "SESS_402",
    "trainee_name": "Rahul Singh",
    "date": "2024-12-09 14:30",
    "result": "PASSED"
  },
```

```
  "metrics": {
    "time_taken": "12:30",
    "cones": [
      {
        "type": "COLD_WARM",
        "user_ppm": 0.6,
        "ideal_ppm": 0.5,
        "status": "ACCURATE" // Green Pin
      },
      {
        "type": "WARM_HOT",
        "user_ppm": 15.0,
        "ideal_ppm": 10.0,
        "status": "DANGEROUS" // Red Pin
      }
    ],
    "triage": {
      "sequence": ["RED", "GREEN", "BLACK"],
      "is_optimal": true
    },
    "failures": []
  },
  "ai_report_markdown": "## Performance Review\n\nOfficer Singh demonstrated **excellent** triage protocols, correctly prioritizing the critical casualty. \n\n### Areas for Improvement\n- **Zone Marking:** The Hot Zone cone was placed at **15 PPM**, which is significantly deep into the danger zone.\n- **Recommendation:** Review SOP-04 regarding gas detector latency."
}
```


---

6. Backend Search & Filter Logic (Implementation Space)

This section is reserved for the backend team to define the SQL queries for the search bars in the Personnel and Reports pages.

Planned Filters:

1. **Date Range:** start_date AND end_date.
2. **Status:** PASSED / FAILED.
3. **Search:** Fuzzy search on trainee_name.

(Backend Team to complete implementation details here during Sprint 2)

End of Specification.
Approved for Frontend Implementation.